

第 6 讲：随机近似

第 6 讲：随机近似

第 5 讲介绍了基于蒙特卡洛估计的第一类无模型强化学习算法，第 7 讲将介绍另一类重要的无模型强化学习算法：时序差分（Temporal-Difference）学习。然而在进入第 7 讲之前，我们需要按下暂停键，为后续内容做更好的准备。这是因为时序差分算法与我们之前学习的算法有很大不同，许多首次接触时序差分算法的读者常常会疑惑：这些算法是如何设计出来的，以及它们为什么能有效工作。事实上，前后章节之间存在一个知识鸿沟：我们之前学习的算法是**非增量式**的，而后续章节的算法是**增量式**的。

本章通过引入随机近似的基本理论来填补这一鸿沟。虽然本章不直接引入任何具体的强化学习算法，但它为后续章节的学习奠定了必要的基础。我们将在第 7 讲中看到，时序差分算法可以被视为一类特殊的随机近似算法。机器学习中广泛使用的随机梯度下降算法也将在本章中介绍。

6.1 动机示例：均值估计

我们首先通过均值估计问题，演示如何将非增量式算法转换为增量式算法。

考虑一个取值于有限集合 $\mathcal{X}$ 的随机变量 X ，目标是估计 $\mathbb{E}[X]$ 。假设我们有一列独立同分布的样本 $\{x_i\}_{i=1}^n$ ， X 的期望值可以通过以下方式近似：

$$\mathbb{E}[X] \approx \bar{x} := \frac{1}{n} \sum_{i=1}^n x_i$$

该近似正是第 5 讲介绍的蒙特卡洛估计的基本思想。根据大数定律， $\bar{x} \rightarrow \mathbb{E}[X]$ ($n \rightarrow \infty$)。

下面我们展示两种计算 $\bar{x}$ 的方法。第一种是**非增量式方法**：先收集所有样本，再计算平均值。这种方法的缺点是：如果样本数量很大，可能需要等待很长时间才能等到所有样本收集完毕。第二种方法可以避免这一缺点，因为它以增量方式计算平均值。具体来说，令

$$w_{k+1} := \frac{1}{k} \sum_{i=1}^k x_i, \quad k = 1, 2, \dots$$

进而

$$w_k = \frac{1}{k-1} \sum_{i=1}^{k-1} x_i, \quad k = 2, 3, \dots$$

则 w_{k+1} 可以用 w_k 表示为：

$$w_{k+1} = \frac{1}{k} \sum_{i=1}^k x_i = \frac{1}{k} \left(\sum_{i=1}^{k-1} x_i + x_k \right) = \frac{1}{k} ((k-1)w_k + x_k) = w_k - \frac{1}{k}(w_k - x_k)$$

因此，我们得到如下**增量式算法**：

$$w_{k+1} = w_k - \frac{1}{k}(w_k - x_k) \tag{1}$$

可以验证：

$$w_1 = x_1, \quad w_2 = x_1, \quad w_3 = \frac{1}{2}(x_1 + x_2), \quad w_4 = \frac{1}{3}(x_1 + x_2 + x_3), \quad \dots, \quad w_{k+1} = \frac{1}{k} \sum_{i=1}^k x_i$$

增量式(1)的优点是：每次收到一个新样本，平均值可以立即更新。该平均值可用于近似 $\bar{x}$ 进而近似 $\mathbb{E}[X]$ 。值得注意的是，由于初始阶段样本不足，近似可能不够精确，但随着样本数量的增加，估计精度会逐步提高。

进一步地，考虑更一般的增量式算法：

$$w_{k+1} = w_k - \alpha_k(w_k - x_k) \quad (2)$$

该算法与(1)相同，只是将系数 $1/k$ 替换为 $\alpha_k > 0$ 。由于 α_k 的具体表达式未给定，我们无法像(1)那样得到 w_k 的显式表达式。但我们将在下一节中证明：如果 $\{\alpha_k\}$ 满足某些温和条件，则 $w_k \rightarrow \mathbb{E}[X]$ ($k \rightarrow \infty$)。在第 7 讲中我们将看到，时序差分算法具有类似（但更复杂）的表达式。

6.2 Robbins-Monro 算法

随机近似（Stochastic Approximation）是指一类用于求解根查找或优化问题的随机迭代算法。与许多其他根查找算法（如基于梯度的算法）相比，随机近似的强大之处在于：它不需要目标函数或其导数的解析表达式。

Robbins-Monro (RM) 算法是随机近似领域的开创性工作。著名的随机梯度下降算法是 RM 算法的一种特殊形式（详见第 6.4 节）。下面我们介绍 RM 算法的细节。

假设我们需要求解方程：

$$g(w) = 0$$

其中 $w \in \mathbb{R}$ 是未知变量， $g: \mathbb{R} \rightarrow \mathbb{R}$ 是一个函数。许多问题都可以表述为根查找问题。例如，若 $J(w)$ 是待优化的目标函数，则该优化问题可转化为求解 $g(w) := \nabla_w J(w) = 0$ 。此外，形如 $g(w) = c$ (c 为常数) 的方程也可以通过将 $g(w) - c$ 视为新函数来转化为上述形式。

然而，我们面临的问题是：函数 g 的表达式是未知的。我们只能获得 $g(w)$ 的带噪观测值：

$$\tilde{g}(w, \eta) = g(w) + \eta$$

其中 $\eta \in \mathbb{R}$ 是观测误差（不一定服从高斯分布）。简而言之，这是一个黑箱系统，只有输入 w 和带噪输出 $\tilde{g}(w, \eta)$ 是已知的。我们的目标是用 w 和 $\tilde{g}$ 求解 $g(w) = 0$ 。

求解 $g(w) = 0$ 的 **RM 算法**为：

$$w_{k+1} = w_k - a_k \tilde{g}(w_k, \eta_k), \quad k = 1, 2, 3, \dots \quad (3)$$

其中 w_k 是对根的第 k 次估计， $\tilde{g}(w_k, \eta_k)$ 是第 k 次带噪观测， a_k 是正系数。可以看出，RM 算法不需要函数的任何信息，只需要输入和输出。

为展示 RM 算法，考虑一个示例： $g(w) = w^3 - 5$ ，真实根为 $5^{1/3} \approx 1.71$ 。假设我们只能观测到输入 w 和输出 $\tilde{g}(w) = g(w) + \eta$ ，其中 η 独立同分布且服从零均值、标准差为 1 的标准正态分布。初始猜测 $w_1 = 0$ ，系数 $a_k = 1/k$ 。即使观测被噪声 η_k 污染，估计值 w_k 仍能收敛到真实根。需要注意，对于 $g(w) = w^3 - 5$ 这一特定函数，初始猜测 w_1 需要适当选取以保证收敛。

6.2.1 收敛性质

为什么 RM 算法能求解 $g(w) = 0$ 的根？我们先通过一个示例说明直观思想，然后给出严格的收敛分析。

考虑 $g(w) = \tanh(w-1)$, 真实根为 $w^* = 1$ 。应用 RM 算法, 取 $w_1 = 3$, $a_k = 1/k$ 。为更清楚地说明收敛原因, 暂时设 $\eta_k \equiv 0$, 因此 $\tilde{g}(w_k, \eta_k) = g(w_k)$ 。此时的 RM 算法即为 $w_{k+1} = w_k - a_k g(w_k)$ 。

该简单示例可以说明 RM 算法收敛的原因:

- 当 $w_k > w^*$ 时, 有 $g(w_k) > 0$, 则 $w_{k+1} = w_k - a_k g(w_k) < w_k$ 。若 $a_k g(w_k)$ 足够小, 则有 $w^* < w_{k+1} < w_k$, 即 w_{k+1} 比 w_k 更接近 w^* 。
- 当 $w_k < w^*$ 时, 有 $g(w_k) < 0$, 则 $w_{k+1} = w_k - a_k g(w_k) > w_k$ 。若 $|a_k g(w_k)|$ 足够小, 则有 $w^* > w_{k+1} > w_k$, 即 w_{k+1} 比 w_k 更接近 w^* 。

无论哪种情况, w_{k+1} 都比 w_k 更接近 w^* , 因此直观上 w_k 收敛到 w^* 。

上述示例是简单的 (观测误差假设为零), 在有随机观测误差的情况下分析收敛性则并非平凡。严格的收敛结果如下。

定理 6.1 Robbins-Monro 定理

在 RM 算法 (3) 中, 若以下条件成立:

1. $0 < c_1 \leq \nabla_w g(w) \leq c_2$ 对所有 w 成立;
2. $\sum_{k=1}^{\infty} a_k = \infty$ 且 $\sum_{k=1}^{\infty} a_k^2 < \infty$;
3. $\mathbb{E}[\eta_k | \mathcal{H}_k] = 0$ 且 $\mathbb{E}[\eta_k^2 | \mathcal{H}_k] < \infty$;

其中 $\mathcal{H}_k = \{w_k, w_{k-1}, \dots\}$, 则 w_k 几乎必然收敛到满足 $g(w^*) = 0$ 的根 w^* 。

该定理的证明推迟到第 6.3.3 节。定理依赖于几乎必然收敛的概念 (参见附录 B)。

定理 6.1 中的三个条件解释如下:

- **第一个条件:** $0 < c_1 \leq \nabla_w g(w)$ 表明 $g(w)$ 是单调递增函数, 这保证了 $g(w) = 0$ 的根存在且唯一。若 $g(w)$ 单调递减, 可简单地将 $-g(w)$ 视为新的单调递增函数。作为一个应用, 若将优化目标函数 $J(w)$ 表述为根查找问题 $g(w) := \nabla_w J(w) = 0$, 则 $g(w)$ 单调递增意味着 $J(w)$ 是凸函数, 这是优化问题中常见的假设。 $\nabla_w g(w) \leq c_2$ 表示 $g(w)$ 的梯度有上界。例如, $g(w) = \tanh(w-1)$ 满足此条件, 但 $g(w) = w^3 - 5$ 不满足。
- **第二个条件:** $\sum_{k=1}^{\infty} a_k^2 < \infty$ 意味着 $\lim_{n \rightarrow \infty} \sum_{k=1}^n a_k^2$ 有上界, 这要求 $a_k \rightarrow 0$ ($k \rightarrow \infty$)。 $\sum_{k=1}^{\infty} a_k = \infty$ 要求 a_k 不能收敛到零太快。这些条件具有有趣的性质, 稍后将详细分析。
- **第三个条件是温和的。** 它不要求观测误差 η_k 服从高斯分布。一个重要特例是 $\{\eta_k\}$ 为独立同分布随机序列, 满足 $\mathbb{E}[\eta_k] = 0$ 且 $\mathbb{E}[\eta_k^2] < \infty$ 。在此情形下, 由于 η_k 与 $\mathcal{H}_k$ 独立, 有 $\mathbb{E}[\eta_k | \mathcal{H}_k] = \mathbb{E}[\eta_k] = 0$ 和 $\mathbb{E}[\eta_k^2 | \mathcal{H}_k] = \mathbb{E}[\eta_k^2]$ 。

下面我们更仔细地分析关于系数 $\{a_k\}$ 的第二个条件。

为什么第二个条件对 RM 算法的收敛很重要?

首先, $\sum_{k=1}^{\infty} a_k^2 < \infty$ 表明 $a_k \rightarrow 0$ ($k \rightarrow \infty$)。假设观测值 $\tilde{g}(w_k, \eta_k)$ 始终有界, 由于 $w_{k+1} - w_k = -a_k \tilde{g}(w_k, \eta_k)$, 若 $a_k \rightarrow 0$, 则 $a_k \tilde{g}(w_k, \eta_k) \rightarrow 0$, 进而 $w_{k+1} - w_k \rightarrow 0$, 表明当 $k \rightarrow \infty$ 时 w_{k+1} 与 w_k 彼此接近。否则, 若 a_k 不收敛, 则 w_k 在 $k \rightarrow \infty$ 时仍可能波动。

其次, $\sum_{k=1}^{\infty} a_k = \infty$ 表明 a_k 不能收敛到零太快。将 $w_2 - w_1 = -a_1 \tilde{g}(w_1, \eta_1)$, $w_3 - w_2 = -a_2 \tilde{g}(w_2, \eta_2)$ 等依次累加:

$$w_{\infty} - w_1 = \sum_{k=1}^{\infty} a_k \tilde{g}(w_k, \eta_k)$$

若 $\sum_{k=1}^{\infty} a_k < \infty$, 则 $|\sum_{k=1}^{\infty} a_k \tilde{g}(w_k, \eta_k)|$ 也有界。设 b 为有限上界, 使得 $|w_{\infty} - w_1| \leq b$ 。若初始猜测 w_1 远离 w^* 以至于 $|w_1 - w^*| > b$, 则不可能有 $w_{\infty} = w^*$ 。这暗示 RM 算法此时无法找到真实解 w^* 。因此, 条件 $\sum_{k=1}^{\infty} a_k = \infty$ 是保证任意初始猜测下收敛的必要条件。

什么样的序列满足这些条件?

一个典型序列是：

$$a_k = \frac{1}{k}$$

一方面，有

$$\lim_{n \rightarrow \infty} \left(\sum_{k=1}^n \frac{1}{k} - \ln n \right) = \gamma$$

其中 $\gamma \approx 0.577$ 称为欧拉-马歇罗尼常数。由于 $\ln n \rightarrow \infty$ ($n \rightarrow \infty$)，因此 $\sum_{k=1}^{\infty} 1/k = \infty$ 。事实上， $H_n = \sum_{k=1}^n 1/k$ 在数论中称为调和数。

另一方面，

$$\sum_{k=1}^{\infty} \frac{1}{k^2} = \frac{\pi^2}{6} < \infty$$

求 $\sum_{k=1}^{\infty} 1/k^2$ 的值即著名的巴塞尔问题。

综上，序列 $\{a_k = 1/k\}$ 满足定理 6.1 的第二个条件。需要注意的是，微小修改（如 $a_k = 1/(k+1)$ 或 $a_k = c_k/k$ 其中 c_k 有界）也保持该条件。

在实际应用中，RM 算法的 a_k 常取为足够小的常数。虽然此时不满足 $\sum a_k^2 < \infty$ （因为 $\sum a_k^2 = \infty$ ），但算法仍可在某种意义下收敛。此外，图 6.3 所示示例中的 $g(x) = x^3 - 5$ 不满足第一个条件，但若初始猜测适当地（而非任意地）选取，RM 算法仍能找到根。

6.2.2 在均值估计中的应用

接下来将 Robbins-Monro 定理应用于第 6.1 节讨论的均值估计问题。回顾更一般的均值估计算法：

$$w_{k+1} = w_k + \alpha_k(x_k - w_k) = w_k - \alpha_k(w_k - x_k)$$

当 $\alpha_k = 1/k$ 时，我们可以得到 $w_{k+1} = (1/k) \sum_{i=1}^k x_i$ 的显式表达式。但当 α_k 取一般值时无法获得显式表达式，收敛分析变得非平凡。我们可以证明，该算法是 RM 算法的一个特例，因此其收敛性自然可得。

具体来说，定义函数：

$$g(w) := w - \mathbb{E}[X]$$

原问题是求 $\mathbb{E}[X]$ 的值。该问题被表述为求解 $g(w) = 0$ 的根查找问题。给定 w 的值，我们可以获得带噪观测 $\tilde{g} := w - x$ ，其中 x 是 X 的一个样本。注意 $\tilde{g}$ 可写为：

$$\tilde{g}(w, \eta) = w - x = w - x + \mathbb{E}[X] - \mathbb{E}[X] = (w - \mathbb{E}[X]) + (\mathbb{E}[X] - x) := g(w) + \eta$$

其中 $\eta := \mathbb{E}[X] - x$ 。

求解该问题的 RM 算法为：

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k) = w_k - \alpha_k(w_k - x_k)$$

这正是算法(2)。因此，由定理 6.1 保证：若 $\sum_{k=1}^{\infty} \alpha_k = \infty$ ， $\sum_{k=1}^{\infty} \alpha_k^2 < \infty$ ，且 $\{x_k\}$ 独立同分布，则 w_k 几乎必然收敛到 $\mathbb{E}[X]$ 。值得强调的是，该收敛性质不依赖于 X 的分布的任何假设。

6.3 Dvoretzky 收敛定理

到目前为止，RM 算法的收敛性尚未得到证明。为此，我们接下来介绍 Dvoretzky 定理，它是随机近似领域的经典结果。该定理可用于分析 RM 算法以及许多强化学习算法的收敛性。

备注

本节数学推导较为密集。对随机算法收敛分析感兴趣的读者建议仔细阅读本节内容，否则可以跳过本节。

定理 6.2 Dvoretzky 定理

考虑随机过程

$$\Delta_{k+1} = (1 - \alpha_k)\Delta_k + \beta_k\eta_k \quad (4)$$

其中 $\{\alpha_k\}_{k=1}^\infty, \{\beta_k\}_{k=1}^\infty, \{\eta_k\}_{k=1}^\infty$ 为随机序列, $\alpha_k \geq 0, \beta_k \geq 0$ 对所有 k 成立。若以下条件满足, 则 Δ_k 几乎必然收敛到零:

1. $\sum_{k=1}^\infty \alpha_k = \infty$, $\sum_{k=1}^\infty \alpha_k^2 < \infty$, 且 $\sum_{k=1}^\infty \beta_k^2 < \infty$, 一致几乎必然成立;
2. $\mathbb{E}[\eta_k | \mathcal{H}_k] = 0$ 且 $\mathbb{E}[\eta_k^2 | \mathcal{H}_k] \leq C$ 几乎必然成立;

其中 $\mathcal{H}_k = \{\Delta_k, \Delta_{k-1}, \dots; \eta_{k-1}, \dots; \alpha_{k-1}, \dots; \beta_{k-1}, \dots\}$ 。

在给出证明之前, 我们先澄清一些细节:

- 在 RM 算法中, 系数序列 $\{\alpha_k\}$ 是确定性的。但 Dvoretzky 定理允许 $\{\alpha_k\}, \{\beta_k\}$ 是依赖于 $\mathcal{H}_k$ 的随机变量, 因此它在 α_k 或 β_k 是 Δ_k 的函数时更为有用。
- 第一个条件中写为”一致几乎必然”, 是因为 α_k 和 β_k 可能是随机变量, 其极限的定义必须在随机意义下理解。第二个条件中也写为”几乎必然”, 因为 $\mathcal{H}_k$ 是随机变量序列而非具体值。因此, $\mathbb{E}[\eta_k | \mathcal{H}_k]$ 和 $\mathbb{E}[\eta_k^2 | \mathcal{H}_k]$ 是随机变量, 此时条件期望的定义是在”几乎必然”意义下的 (参见附录 B)。
- 定理 6.2 的陈述与原始文献略有不同: 定理 6.2 在第一个条件中不要求 $\sum \beta_k = \infty$ 。当 $\sum \beta_k < \infty$ 时, 特别是在极端情况 $\beta_k = 0$ (对所有 k), 序列仍然可以收敛。

6.3.1 Dvoretzky 定理的证明

Dvoretzky 定理最早的证明于 1956 年给出。我们下面给出一个基于拟鞅 (quasimartingale) 的证明。借助拟鞅的收敛结果, Dvoretzky 定理的证明是直接的。关于拟鞅的更多信息可参见附录 C。

证明

令 $h_k := \Delta_k^2$ 。则:

$$\begin{aligned} h_{k+1} - h_k &= \Delta_{k+1}^2 - \Delta_k^2 \\ &= (\Delta_{k+1} - \Delta_k)(\Delta_{k+1} + \Delta_k) \\ &= (-\alpha_k\Delta_k + \beta_k\eta_k)[(2 - \alpha_k)\Delta_k + \beta_k\eta_k] \\ &= -\alpha_k(2 - \alpha_k)\Delta_k^2 + \beta_k^2\eta_k^2 + 2(1 - \alpha_k)\beta_k\eta_k\Delta_k \end{aligned}$$

对上式两边取条件期望:

$$\mathbb{E}[h_{k+1} - h_k | \mathcal{H}_k] = \mathbb{E}[-\alpha_k(2 - \alpha_k)\Delta_k^2 | \mathcal{H}_k] + \mathbb{E}[\beta_k^2\eta_k^2 | \mathcal{H}_k] + \mathbb{E}[2(1 - \alpha_k)\beta_k\eta_k\Delta_k | \mathcal{H}_k] \quad (5)$$

首先, 由于 Δ_k 包含于 $\mathcal{H}_k$ 中且由 $\mathcal{H}_k$ 确定, 可以从期望中提出 (参见引理 B.1 的性质 (e))。其次, 考虑 α_k, β_k 由 $\mathcal{H}_k$ 确定的简单情形 (例如当 $\{\alpha_k\}$ 和 $\{\beta_k\}$ 是 Δ_k 的函数或是确定序列时), 它们也可以从期望中提出。因此, (5)变为:

$$\mathbb{E}[h_{k+1} - h_k | \mathcal{H}_k] = -\alpha_k(2 - \alpha_k)\Delta_k^2 + \beta_k^2\mathbb{E}[\eta_k^2 | \mathcal{H}_k] + 2(1 - \alpha_k)\beta_k\Delta_k\mathbb{E}[\eta_k | \mathcal{H}_k]$$

对于第一项, 由于 $\sum_{k=1}^{\infty} \alpha_k^2 < \infty$ 蕴含 $\alpha_k \rightarrow 0$ 几乎必然, 存在有限 n 使得对所有 $k \geq n$, 有 $\alpha_k \leq 1$ 几乎必然。不失一般性, 下文仅考虑 $\alpha_k \leq 1$ 的情形。此时 $-\alpha_k(2 - \alpha_k)\Delta_k^2 \leq 0$ 。对于第二项, 由假设有 $\beta_k^2 \mathbb{E}[\eta_k^2 | \mathcal{H}_k] \leq \beta_k^2 C$ 。第三项为零, 因为假设 $\mathbb{E}[\eta_k | \mathcal{H}_k] = 0$ 。因此:

$$\mathbb{E}[h_{k+1} - h_k | \mathcal{H}_k] = -\alpha_k(2 - \alpha_k)\Delta_k^2 + \beta_k^2 \mathbb{E}[\eta_k^2 | \mathcal{H}_k] \leq \beta_k^2 C$$

进而:

$$\sum_{k=1}^{\infty} \mathbb{E}[h_{k+1} - h_k | \mathcal{H}_k] \leq \sum_{k=1}^{\infty} \beta_k^2 C < \infty$$

最后一个不等式由条件 $\sum_{k=1}^{\infty} \beta_k^2 < \infty$ 得到。于是, 根据附录 C 的拟鞅收敛定理, h_k 几乎必然收敛。

下面确定 Δ_k 收敛到何值。由上述不等式可得:

$$\sum_{k=1}^{\infty} \alpha_k(2 - \alpha_k)\Delta_k^2 = \sum_{k=1}^{\infty} \beta_k^2 \mathbb{E}[\eta_k^2 | \mathcal{H}_k] - \sum_{k=1}^{\infty} \mathbb{E}[h_{k+1} - h_k | \mathcal{H}_k]$$

右端第一项由假设有界, 第二项也有界 (因为 h_k 收敛, $h_{k+1} - h_k$ 可求和)。因此左端 $\sum_{k=1}^{\infty} \alpha_k(2 - \alpha_k)\Delta_k^2$ 有界。由于考虑 $\alpha_k \leq 1$ 的情形, 有:

$$\infty > \sum_{k=1}^{\infty} \alpha_k(2 - \alpha_k)\Delta_k^2 \geq \sum_{k=1}^{\infty} \alpha_k \Delta_k^2 \geq 0$$

因此 $\sum_{k=1}^{\infty} \alpha_k \Delta_k^2$ 有界。由于 $\sum_{k=1}^{\infty} \alpha_k = \infty$, 必须有 $\Delta_k \rightarrow 0$ 几乎必然。

6.3.2 在均值估计中的应用

虽然均值估计算法 $w_{k+1} = w_k + \alpha_k(x_k - w_k)$ 已通过 RM 定理得证, 但我们展示其收敛性也可以直接由 Dvoretzky 定理证明。

证明

令 $w^* = \mathbb{E}[X]$ 。均值估计算法 $w_{k+1} = w_k + \alpha_k(x_k - w_k)$ 可重写为:

$$w_{k+1} - w^* = w_k - w^* + \alpha_k(x_k - w^* + w^* - w_k)$$

令 $\Delta := w - w^*$, 则有:

$$\Delta_{k+1} = \Delta_k + \alpha_k(x_k - w^* - \Delta_k) = (1 - \alpha_k)\Delta_k + \alpha_k \underbrace{(x_k - w^*)}_{\eta_k}$$

由于 $\{x_k\}$ 独立同分布, 有 $\mathbb{E}[x_k | \mathcal{H}_k] = \mathbb{E}[x_k] = w^*$ 。因此:

$$\mathbb{E}[\eta_k | \mathcal{H}_k] = \mathbb{E}[x_k - w^* | \mathcal{H}_k] = 0$$

$$\mathbb{E}[\eta_k^2 | \mathcal{H}_k] = \mathbb{E}[x_k^2 | \mathcal{H}_k] - (w^*)^2 = \mathbb{E}[x_k^2] - (w^*)^2$$

若 x_k 的方差有限, 则上式有界。根据 Dvoretzky 定理, $\Delta_k \rightarrow 0$ 进而 $w_k \rightarrow w^* = \mathbb{E}[X]$ 几乎必然。

6.3.3 在 Robbins-Monro 定理中的应用

现在我们利用 Dvoretzky 定理来证明 Robbins-Monro 定理。

证明

RM 算法的目标是求 $g(w) = 0$ 的根。设根为 w^* 使得 $g(w^*) = 0$ 。RM 算法为：

$$w_{k+1} = w_k - a_k \tilde{g}(w_k, \eta_k) = w_k - a_k [g(w_k) + \eta_k]$$

则：

$$w_{k+1} - w^* = w_k - w^* - a_k [g(w_k) - g(w^*) + \eta_k]$$

由中值定理， $g(w_k) - g(w^*) = \nabla_w g(w'_k)(w_k - w^*)$ ，其中 $w'_k \in [w_k, w^*]$ 。令 $\Delta_k := w_k - w^*$ ，上式变为：

$$\begin{aligned} \Delta_{k+1} &= \Delta_k - a_k [\nabla_w g(w'_k)(w_k - w^*) + \eta_k] \\ &= \Delta_k - a_k \nabla_w g(w'_k) \Delta_k + a_k (-\eta_k) \\ &= [1 - \underbrace{a_k \nabla_w g(w'_k)}_{\alpha_k}] \Delta_k + a_k (-\eta_k) \end{aligned}$$

注意由假设有 $\nabla_w g(w)$ 有界： $0 < c_1 \leq \nabla_w g(w) \leq c_2$ 。由于 $\sum_{k=1}^{\infty} a_k = \infty$ 且 $\sum_{k=1}^{\infty} a_k^2 < \infty$ (假设条件)，可知 $\sum_{k=1}^{\infty} \alpha_k = \infty$ 且 $\sum_{k=1}^{\infty} \alpha_k^2 < \infty$ 。于是 Dvoretzky 定理的所有条件均满足，因此 Δ_k 几乎必然收敛到零。

RM 定理的证明展示了 Dvoretzky 定理的强大之处。特别地，证明中的 α_k 是一个依赖于 w_k 的随机序列而非确定性序列，此时 Dvoretzky 定理仍然适用。

6.3.4 Dvoretzky 定理的推广

下面将 Dvoretzky 定理推广到可以处理多个变量的更一般形式。该推广定理可用于分析随机迭代算法（如 Q-learning）的收敛性。

定理 6.3 推广的 Dvoretzky 定理

考虑定义在有限实数集合 $\mathcal{S}$ 上的随机过程：

$$\Delta_{k+1}(s) = (1 - \alpha_k(s))\Delta_k(s) + \beta_k(s)\eta_k(s)$$

若以下条件对 $s \in \mathcal{S}$ 成立，则对每个 $s \in \mathcal{S}$ ， $\Delta_k(s)$ 几乎必然收敛到零：

1. $\sum_k \alpha_k(s) = \infty$ ， $\sum_k \alpha_k^2(s) < \infty$ ， $\sum_k \beta_k^2(s) < \infty$ ，且 $\mathbb{E}[\beta_k(s) | \mathcal{H}_k] \leq \mathbb{E}[\alpha_k(s) | \mathcal{H}_k]$ 一致几乎必然成立；
2. $\|\mathbb{E}[\eta_k(s) | \mathcal{H}_k]\|_{\infty} \leq \gamma \|\Delta_k\|_{\infty}$ ，其中 $\gamma \in (0, 1)$ ；
3. $\text{var}[\eta_k(s) | \mathcal{H}_k] \leq C(1 + \|\Delta_k(s)\|_{\infty})^2$ ，其中 C 为常数。

其中 $\mathcal{H}_k = \{\Delta_k, \Delta_{k-1}, \dots; \eta_{k-1}, \dots; \alpha_{k-1}, \dots; \beta_{k-1}, \dots\}$ 表示历史信息。 $\|\Delta_k\|_{\infty}$ 表示最大范数。

关于该定理的一些注记：

- 变量 s 可视为索引。在强化学习的语境中，它表示一个状态或状态-动作对。最大范数 $\|\Delta_k\|_{\infty}$ 定义在集合上，与向量的 L_{∞} 范数类似但有所不同。具体地， $\|\mathbb{E}[\eta_k(s) | \mathcal{H}_k]\|_{\infty} := \max_{s \in \mathcal{S}} |\mathbb{E}[\eta_k(s) | \mathcal{H}_k]|$ ， $\|\Delta_k(s)\|_{\infty} := \max_{s \in \mathcal{S}} |\Delta_k(s)|$ 。
- 该定理比 Dvoretzky 定理更一般。首先，它可以处理多变量情形（通过最大范数运算），这对具

有多个状态的强化学习问题很重要。其次，Dvoretzky 定理要求 $\mathbb{E}[\eta_k(s) \mid \mathcal{H}_k] = 0$ 和 $\text{var}[\eta_k(s) \mid \mathcal{H}_k] \leq C$ ，而该定理只要求期望和方差被误差 Δ_k 所界定。

- 需要指出， $\Delta(s)$ 对所有 $s \in \mathcal{S}$ 收敛要求条件对每个 $s \in \mathcal{S}$ 都成立。因此，在应用该定理证明强化学习算法收敛时，需要验证条件对每个状态（或状态-动作对）均成立。

6.4 随机梯度下降

本节介绍机器学习领域广泛使用的随机梯度下降（Stochastic Gradient Descent, SGD）算法。我们将看到，SGD 是 RM 算法的一个特例，而均值估计算法是 SGD 的一个特例。

考虑如下优化问题：

$$\min_w J(w) = \mathbb{E}[f(w, X)] \quad (6)$$

其中 w 是待优化的参数， X 是随机变量。期望是关于 X 计算的。 w 和 X 可以是标量或向量，函数 $f(\cdot)$ 为标量。

求解(6)的直接方法是**梯度下降**。具体地， $\mathbb{E}[f(w, X)]$ 的梯度为 $\nabla_w \mathbb{E}[f(w, X)] = \mathbb{E}[\nabla_w f(w, X)]$ ，梯度下降算法为：

$$w_{k+1} = w_k - \alpha_k \nabla_w J(w_k) = w_k - \alpha_k \mathbb{E}[\nabla_w f(w_k, X)] \quad (7)$$

在 f 为凸函数等一些温和条件下，该梯度下降算法能够找到最优解 w^* 。

梯度下降算法需要期望值 $\mathbb{E}[\nabla_w f(w_k, X)]$ 。一种获取期望值的方法是基于 X 的概率分布，然而在实际中该分布通常是未知的。另一种方法是收集 X 的大量独立同分布样本 $\{x_i\}_{i=1}^n$ ，用样本均值近似期望：

$$\mathbb{E}[\nabla_w f(w_k, X)] \approx \frac{1}{n} \sum_{i=1}^n \nabla_w f(w_k, x_i)$$

则(7)变为：

$$w_{k+1} = w_k - \frac{\alpha_k}{n} \sum_{i=1}^n \nabla_w f(w_k, x_i) \quad (8)$$

(8)的一个问题是每次迭代需要所有样本。在实际中，若样本是逐个收集的，则希望每收到一个样本就更新一次 w 。为此，可使用如下算法：

$$w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k) \quad (9)$$

其中 x_k 是第 k 步收集到的样本。这就是著名的**随机梯度下降算法**。该算法称为“随机”是因为它依赖于随机样本 $\{x_k\}$ 。

与梯度下降算法(7)相比，SGD 用随机梯度 $\nabla_w f(w_k, x_k)$ 替代了真实梯度 $\mathbb{E}[\nabla_w f(w, X)]$ 。由于 $\nabla_w f(w_k, x_k) \neq \mathbb{E}[\nabla_w f(w, X)]$ ，这样的替代是否仍能保证 $w_k \rightarrow w^*$ ($k \rightarrow \infty$)？答案是肯定的。下面给出直观解释，严格证明推迟到第 6.4.5 节。

具体来说，由于：

$$\nabla_w f(w_k, x_k) = \mathbb{E}[\nabla_w f(w_k, X)] + (\nabla_w f(w_k, x_k) - \mathbb{E}[\nabla_w f(w_k, X)]) := \mathbb{E}[\nabla_w f(w_k, X)] + \eta_k$$

SGD 算法(9)可重写为：

$$w_{k+1} = w_k - \alpha_k \mathbb{E}[\nabla_w f(w_k, X)] - \alpha_k \eta_k$$

因此，SGD 算法与常规梯度下降算法相同，只是多了一个扰动项 $\alpha_k \eta_k$ 。由于 $\{x_k\}$ 独立同分布，有 $\mathbb{E}_{x_k}[\nabla_w f(w_k, x_k)] = \mathbb{E}_X[\nabla_w f(w_k, X)]$ 。因此：

$$\mathbb{E}[\eta_k] = \mathbb{E}[\nabla_w f(w_k, x_k) - \mathbb{E}[\nabla_w f(w_k, X)]] = \mathbb{E}_{x_k}[\nabla_w f(w_k, x_k)] - \mathbb{E}_X[\nabla_w f(w_k, X)] = 0$$

因此扰动项 η_k 均值为零，这直观地暗示它可能不会损害收敛性质。SGD 收敛性的严格证明在第 6.4.5 节给出。

6.4.1 在均值估计中的应用

下面应用 SGD 分析均值估计问题，并证明均值估计算法(2)是 SGD 算法的一个特例。为此，将均值估计问题表述为优化问题：

$$\min_w J(w) = \mathbb{E} \left[\frac{1}{2} \|w - X\|^2 \right] := \mathbb{E}[f(w, X)] \quad (10)$$

其中 $f(w, X) = \|w - X\|^2/2$ ，梯度为 $\nabla_w f(w, X) = w - X$ 。可以验证：通过求解 $\nabla_w J(w) = 0$ 可得最优解 $w^* = \mathbb{E}[X]$ 。因此该优化问题等价于均值估计问题。

- 求解(10)的梯度下降算法为：

$$w_{k+1} = w_k - \alpha_k \nabla_w J(w_k) = w_k - \alpha_k \mathbb{E}[\nabla_w f(w_k, X)] = w_k - \alpha_k \mathbb{E}[w_k - X]$$

该梯度下降算法不可用，因为右端的 $\mathbb{E}[w_k - X]$ （或 $\mathbb{E}[X]$ ）未知。

- 求解(10)的 SGD 算法为：

$$w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k) = w_k - \alpha_k (w_k - x_k)$$

其中 x_k 是第 k 步获得的样本。注意，该 SGD 算法正是(2)中的迭代均值估计算法。因此，(2)是专门为求解均值估计问题而设计的 SGD 算法。

6.4.2 SGD 的收敛模式

SGD 算法的核心思想是用随机梯度替代真实梯度。然而，由于随机梯度是随机的，自然会问：SGD 的收敛速度是否缓慢或不稳定？幸运的是，SGD 通常可以高效收敛。一个有趣的收敛模式是：当估计值 w_k 远离最优解 w^* 时，SGD 的行为类似于常规梯度下降算法；只有当 w_k 接近 w^* 时，SGD 的收敛才表现出更多随机性。

下面给出该模式的分析和一个示例。

分析：随机梯度与真实梯度之间的相对误差为：

$$\delta_k := \frac{|\nabla_w f(w_k, x_k) - \mathbb{E}[\nabla_w f(w_k, X)]|}{|\mathbb{E}[\nabla_w f(w_k, X)]|}$$

为简化起见，考虑 w 和 $\nabla_w f(w, x)$ 均为标量的情形。由于 w^* 是最优解，有 $\mathbb{E}[\nabla_w f(w^*, X)] = 0$ 。相对误差可重写为：

$$\delta_k = \frac{|\nabla_w f(w_k, x_k) - \mathbb{E}[\nabla_w f(w_k, X)]|}{|\mathbb{E}[\nabla_w f(w_k, X)] - \mathbb{E}[\nabla_w f(w^*, X)]|} = \frac{|\nabla_w f(w_k, x_k) - \mathbb{E}[\nabla_w f(w_k, X)]|}{|\mathbb{E}[\nabla_w^2 f(\tilde{w}_k, X)(w_k - w^*)]|}$$

其中最后一个等式由中值定理得到， $\tilde{w}_k \in [w_k, w^*]$ 。

假设 f 是严格凸的，即 $\nabla_w^2 f \geq c > 0$ 对所有 w, X 成立。则分母变为：

$$|\mathbb{E}[\nabla_w^2 f(\tilde{w}_k, X)(w_k - w^*)]| = |\mathbb{E}[\nabla_w^2 f(\tilde{w}_k, X)]| \cdot |w_k - w^*| \geq c|w_k - w^*|$$

代入上式得：

$$\delta_k \leq \frac{|\nabla_w f(w_k, x_k) - \mathbb{E}[\nabla_w f(w_k, X)]|}{c|w_k - w^*|}$$

该不等式揭示了 SGD 的一个有趣收敛模式：相对误差 δ_k 与 $|w_k - w^*|$ 成反比。因此，当 $|w_k - w^*|$ 较大时， δ_k 较小，此时 SGD 的行为类似于梯度下降算法， w_k 快速收敛到 w^* 。当 w_k 接近 w^* 时，相对误差 δ_k 可能较大，收敛表现出更多随机性。

示例：演示上述分析的绝佳例子是均值估计问题。考虑(10)中的均值估计问题。当 w 和 X 均为标量时， $f(w, X) = |w - X|^2/2$ ，有：

$$\nabla_w f(w, x_k) = w - x_k, \quad \mathbb{E}[\nabla_w f(w, x_k)] = w - \mathbb{E}[X] = w - w^*$$

因此相对误差为：

$$\delta_k = \frac{|(w_k - x_k) - (w_k - \mathbb{E}[X])|}{|w_k - w^*|} = \frac{|\mathbb{E}[X] - x_k|}{|w_k - w^*|}$$

相对误差的表达式清楚地表明 δ_k 与 $|w_k - w^*|$ 成反比。当 w_k 远离 w^* 时相对误差较小，SGD 的行为类似于梯度下降。此外，由于 δ_k 与 $|\mathbb{E}[X] - x_k|$ 成正比， δ_k 的均值与 X 的方差成正比。

仿真结果如图 6.5 所示： $X \in \mathbb{R}^2$ 表示平面中的随机位置，其分布在以原点为中心的正方形区域内均匀分布， $\mathbb{E}[X] = 0$ 。均值估计基于 100 个独立同分布样本。虽然均值的初始猜测远离真实值，但 SGD 估计迅速接近原点邻域。当估计接近原点时，收敛过程表现出一定的随机性。

6.4.3 SGD 的确定性形式

SGD 的(9)形式涉及随机变量。我们常常会遇到不含任何随机变量的确定性形式的 SGD。

具体来说，考虑一组实数 $\{x_i\}_{i=1}^n$ (x_i 不必是任何随机变量的样本)，待求解的优化问题为最小化平均值：

$$\min_w J(w) = \frac{1}{n} \sum_{i=1}^n f(w, x_i)$$

其中 $f(w, x_i)$ 是参数化函数， w 是待优化参数。求解该问题的梯度下降算法为：

$$w_{k+1} = w_k - \alpha_k \nabla_w J(w_k) = w_k - \frac{\alpha_k}{n} \sum_{i=1}^n \nabla_w f(w_k, x_i)$$

假设集合 $\{x_i\}_{i=1}^n$ 很大，每次只能获取一个数，此时更适合以增量方式更新 w_k ：

$$w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k) \quad (11)$$

需注意，此处的 x_k 是第 k 步获取的数，而非集合 $\{x_i\}_{i=1}^n$ 中的第 k 个元素。

算法(11)与 SGD 非常相似，但其问题表述有微妙的差异——不涉及任何随机变量或期望值。这会引出许多问题：这是 SGD 算法吗？应如何使用有限集合 $\{x_i\}_{i=1}^n$ ？是否应按某种顺序排序后逐个使用，还是应从集合中随机采样？

对上述问题的简要回答是：虽然上述表述中不含随机变量，但可以通过引入随机变量将确定性表述转化为随机表述。具体地，令 X 是定义在集合 $\{x_i\}_{i=1}^n$ 上的随机变量，假设其概率分布为均匀分布： $\mathbb{P}(X = x_i) = 1/n$ 。则确定性优化问题变为随机优化问题：

$$\min_w J(w) = \frac{1}{n} \sum_{i=1}^n f(w, x_i) = \mathbb{E}[f(w, X)]$$

上式中最后一个等号是严格等号而非近似。因此，(11)确实是 SGD，且若 x_k 是从 $\{x_i\}_{i=1}^n$ 中均匀且独立地采样得到，则估计值收敛。注意 x_k 可能由于随机采样的原因重复取到 $\{x_i\}_{i=1}^n$ 中的同一个数。

6.4.4 BGD、SGD 与 MBGD

SGD 每次迭代仅使用一个样本，接下来介绍小批量梯度下降 (Mini-Batch Gradient Descent, MBGD)，它每次迭代使用更多样本。当每次迭代使用所有样本时，算法称为批量梯度下降 (Batch Gradient Descent, BGD)。

具体来说，假设我们希望找到最小化 $J(w) = \mathbb{E}[f(w, X)]$ 的最优解，给定 X 的一组随机样本

$\{x_i\}_{i=1}^n$ 。求解该问题的 BGD、SGD 和 MBGD 算法分别为：

$$w_{k+1} = w_k - \frac{\alpha_k}{n} \sum_{i=1}^n \nabla_w f(w_k, x_i) \quad (\text{BGD})$$

$$w_{k+1} = w_k - \frac{\alpha_k}{m} \sum_{j \in \mathcal{I}_k} \nabla_w f(w_k, x_j) \quad (\text{MBGD})$$

$$w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k) \quad (\text{SGD})$$

在 BGD 算法中，每次迭代使用全部 n 个样本。当 n 较大时， $(1/n) \sum_{i=1}^n \nabla_w f(w_k, x_i)$ 接近真实梯度 $\mathbb{E}[\nabla_w f(w_k, X)]$ 。在 MBGD 算法中， $\mathcal{I}_k$ 是第 k 步从 $\{1, \dots, n\}$ 中获取的子集，大小为 $|\mathcal{I}_k| = m$ ， $\mathcal{I}_k$ 中的样本也假设为独立同分布。在 SGD 算法中， x_k 是第 k 步从 $\{x_i\}_{i=1}^n$ 中随机采样的一个样本。

MBGD 可视为 SGD 与 BGD 之间的中间版本。与 SGD 相比，MBGD 的随机性更小（因为它用多个样本而非仅一个样本）。与 BGD 相比，MBGD 不需要在每次迭代中使用所有样本，更加灵活。若 $m = 1$ ，MBGD 退化为 SGD。但若 $m = n$ ，MBGD 未必等同于 BGD，因为 MBGD 使用 n 个随机抽取的样本（可能有重复），而 BGD 使用所有 n 个不同的数。

MBGD 的收敛速度通常快于 SGD。这是因为 SGD 用 $\nabla_w f(w_k, x_k)$ 近似真实梯度，而 MBGD 用 $(1/m) \sum_{j \in \mathcal{I}_k} \nabla_w f(w_k, x_j)$ 进行近似，由于随机性被平均掉，后者更接近真实梯度。MBGD 算法的收敛性可类似 SGD 情形证明。

演示上述分析的绝佳例子是均值估计问题。具体来说，给定若干数 $\{x_i\}_{i=1}^n$ ，目标是计算均值 $\bar{x} = \sum_{i=1}^n x_i / n$ 。该问题可等价表述为如下优化问题：

$$\min_w J(w) = \frac{1}{2n} \sum_{i=1}^n \|w - x_i\|^2$$

其最优解为 $w^* = \bar{x}$ 。求解该问题的三种算法分别为：

$$w_{k+1} = w_k - \frac{\alpha_k}{n} \sum_{i=1}^n (w_k - x_i) = w_k - \alpha_k (w_k - \bar{x}) \quad (\text{BGD})$$

$$w_{k+1} = w_k - \frac{\alpha_k}{m} \sum_{j \in \mathcal{I}_k} (w_k - x_j) = w_k - \alpha_k (w_k - \bar{x}_k^{(m)}) \quad (\text{MBGD})$$

$$w_{k+1} = w_k - \alpha_k (w_k - x_k) \quad (\text{SGD})$$

其中 $\bar{x}_k^{(m)} = \sum_{j \in \mathcal{I}_k} x_j / m$ 。进一步，若 $\alpha_k = 1/k$ ，上述方程可解为：

$$w_{k+1} = \frac{1}{k} \sum_{j=1}^k \bar{x} = \bar{x} \quad (\text{BGD})$$

$$w_{k+1} = \frac{1}{k} \sum_{j=1}^k \bar{x}_j^{(m)} \quad (\text{MBGD})$$

$$w_{k+1} = \frac{1}{k} \sum_{j=1}^k x_j \quad (\text{SGD})$$

可以看出，BGD 在每一步给出的估计值恰好就是最优解 $w^* = \bar{x}$ 。MBGD 收敛到均值比 SGD 更快，因为 $\bar{x}_k^{(m)}$ 本身已经是一个平均值。

图 6.5 中的仿真示例演示了 MBGD 的收敛情况。取 $\alpha_k = 1/k$ ，不同小批量大小的 MBGD 算法均能收敛到均值。 $m = 50$ 的情形收敛最快，而 $m = 1$ 的 SGD 最慢，这与上述分析一致。尽管如此，SGD 的收敛速度仍然很快，尤其是当 w_k 远离 w^* 时。

6.4.5 SGD 的收敛性

SGD 收敛性的严格证明如下。

定理 6.4 SGD 的收敛性

对于 SGD 算法(9)，若以下条件满足，则 w_k 几乎必然收敛到 $\nabla_w \mathbb{E}[f(w, X)] = 0$ 的根：

1. $0 < c_1 \leq \nabla_w^2 f(w, X) \leq c_2$;
2. $\sum_{k=1}^{\infty} a_k = \infty$ 且 $\sum_{k=1}^{\infty} a_k^2 < \infty$;
3. $\{x_k\}_{k=1}^{\infty}$ 独立同分布。

定理 6.4 中的三个条件讨论如下：

- 条件 (a) 关于 f 的凸性，要求 f 的曲率有上下界。此处 w 为标量， $\nabla_w^2 f(w, X)$ 也是标量。该条件可推广到向量情形：当 w 为向量时， $\nabla_w^2 f(w, X)$ 即著名的 Hessian 矩阵。
- 条件 (b) 与 RM 算法的条件类似。事实上，SGD 算法是 RM 算法的一个特例（见下方方框 6.1 中的证明）。在实际中， a_k 常取为足够小的常数，虽然此时不满足条件 (b)，但算法仍可在某种意义上收敛。
- 条件 (c) 是常见的要求。

证明

下面证明 SGD 算法是 RM 算法的一个特例，从而 SGD 的收敛性自然由 RM 定理得到。SGD 要求解的问题是 $\min_w J(w) = \mathbb{E}[f(w, X)]$ 。该问题可转化为根查找问题：求 $\nabla_w J(w) = \mathbb{E}[\nabla_w f(w, X)] = 0$ 的根。令

$$g(w) = \nabla_w J(w) = \mathbb{E}[\nabla_w f(w, X)]$$

则 SGD 的目标是求 $g(w) = 0$ 的根，这正是 RM 算法解决的问题。我们可观测的量为 $\tilde{g} = \nabla_w f(w, x)$ ，其中 x 是 X 的一个样本。注意 $\tilde{g}$ 可重写为：

$$\tilde{g}(w, \eta) = \nabla_w f(w, x) = \mathbb{E}[\nabla_w f(w, X)] + \underbrace{\nabla_w f(w, x) - \mathbb{E}[\nabla_w f(w, X)]}_{\eta}$$

则求解 $g(w) = 0$ 的 RM 算法为：

$$w_{k+1} = w_k - a_k \tilde{g}(w_k, \eta_k) = w_k - a_k \nabla_w f(w_k, x_k)$$

这正是 SGD 算法(9)。因此 SGD 算法是 RM 算法的一个特例。下面验证定理 6.1 中的三个条件均满足，则 SGD 的收敛性自然由定理 6.1 得到。

- 由于 $\nabla_w g(w) = \nabla_w \mathbb{E}[\nabla_w f(w, X)] = \mathbb{E}[\nabla_w^2 f(w, X)]$ ，由 $c_1 \leq \nabla_w^2 f(w, X) \leq c_2$ 可得 $c_1 \leq \nabla_w g(w) \leq c_2$ 。因此定理 6.1 的第一个条件满足。
- 定理 6.1 的第二个条件与本定理的第二个条件相同。
- 定理 6.1 的第三个条件要求 $\mathbb{E}[\eta_k | \mathcal{H}_k] = 0$ 且 $\mathbb{E}[\eta_k^2 | \mathcal{H}_k] < \infty$ 。由于 $\{x_k\}$ 独立同分布，对所有 k 有 $\mathbb{E}_{x_k}[\nabla_w f(w, x_k)] = \mathbb{E}[\nabla_w f(w, X)]$ 。因此：

$$\begin{aligned} \mathbb{E}[\eta_k | \mathcal{H}_k] &= \mathbb{E}[\nabla_w f(w_k, x_k) - \mathbb{E}[\nabla_w f(w_k, X)] | \mathcal{H}_k] \\ &= \mathbb{E}_{x_k}[\nabla_w f(w_k, x_k)] - \mathbb{E}[\nabla_w f(w_k, X)] = 0 \end{aligned}$$

类似可证，若对任意 w 和任意 x 有 $|\nabla_w f(w, x)| < \infty$ ，则 $\mathbb{E}[\eta_k^2 | \mathcal{H}_k] < \infty$ 。由于定理 6.1 的三个条件均满足，SGD 算法的收敛性得证。

6.5 本章小结

本章没有引入新的强化学习算法，而是介绍了随机近似的基础知识，包括 RM 算法和 SGD 算法。与许多其他根查找算法相比，RM 算法不需要目标函数或其导数的表达式。我们证明了 SGD 算法是 RM 算法的一个特例。此外，均值估计问题是贯穿本章反复讨论的重要问题：均值估计算法(2)是本书引入的第一个随机迭代算法，我们证明它是 SGD 算法的一个特例。在第 7 讲中我们将看到，时序差分学习算法具有类似的表达式。最后，”随机近似”这一名称由 Robbins 和 Monro 于 1951 年首次使用。

6.6 常见问题解答

- **Q: 什么是随机近似?**

A: 随机近似是指一类用于求解根查找或优化问题的随机迭代算法。

- **Q: 为什么需要学习随机近似?**

A: 因为第 7 讲将介绍的时序差分强化学习算法可视为随机近似算法。掌握了本章的知识，我们在首次见到这些算法时就不会感到突兀。

- **Q: 为什么本章频繁讨论均值估计问题?**

A: 因为状态价值和动作价值被定义为随机变量的均值，第 7 讲介绍的时序差分学习算法类似于用于均值估计的随机近似算法。

- **Q: RM 算法相对于其他根查找算法有什么优势?**

A: 与许多其他根查找算法相比，RM 算法的强大之处在于它不需要目标函数或其导数的表达式。因此它是一种只需要目标函数输入和输出的黑箱技术。著名的 SGD 算法是 RM 算法的一种特殊形式。

- **Q: SGD 的基本思想是什么?**

A: SGD 旨在求解涉及随机变量的优化问题。当给定随机变量的概率分布未知时，SGD 可以仅通过样本来求解优化问题。数学上，SGD 算法可以通过将梯度下降算法中表示为期望的真实梯度替换为随机梯度而得到。

- **Q: SGD 能快速收敛吗?**

A: SGD 有一个有趣的收敛模式：若估计值远离最优解，则收敛过程较快；当估计值接近最优解时，随机梯度的随机性变得显著，收敛速度下降。

- **Q: 什么是 MBGD? 它相对于 SGD 和 BGD 有什么优势?**

A: MBGD 可视为 SGD 与 BGD 之间的中间版本。与 SGD 相比，它的随机性更小，因为它使用更多样本而非仅一个样本。与 BGD 相比，它不需要使用所有样本，更加灵活。